FSA-AstraInstall Automation V3.0.147 (2025.12.03)

Компания: ООО "НПА Вира-Реалтайм"
Разработчик: @FoksSegr & AI Assistant (@LLM)

📋 Оглавление

1. Описание проекта
2. Основные возможности
3. Архитектура системы
4. Модульная структура проекта
5. Графический интерфейс
6. Система сборки и развертывания
7. Управление компонентами
8. Использование
9. Технические требования
10. Поддержка и документация

📖 Описание проекта

FSA-AstraInstall — комплексная автоматизированная система для управления Linux-системами, обеспечивающая автоматическое обновление операционных систем, установку и управление компонентами (Wine, среды разработки, приложения) с возможностью переустановки и удаления в процессе эксплуатации.

Ключевые характеристики

Параметр	Значение
Версия	V3.0.146 (коммит #145)
Архитектура	UniversalProcessRunner v2.0.0
Размер бинарника	11.8 MB (автономный исполняемый файл)
Размер Python-файла	1.6 MB (29,791 строк)
Количество классов	33
Основной скрипт	995 строк
GUI менеджер пакетов	6,298 строк

Совместимость

Платформа	Версия	Статус
Astra Linux	1.7.x (Смоленск)	✅ Полная поддержка

Платформа	Версия	Статус
Astra Linux	1.8.x (Смоленск)	✅ Полная поддержка
Ubuntu	20.04+	✅ Полная поддержка
Debian	10+ (Buster)	✅ Полная поддержка

Требования: GLIBC 2.28+ (автоматически выполняется на всех поддерживаемых системах)

🎯 Основные возможности

Автоматизация обновления системы

- ✅ **Автоматическое обновление Linux систем** из доступных репозиториев
- ✅ **Усиленное автоподтверждение запросов** — многоуровневая система автоответов:
  - Переменные окружения (DEBIAN_FRONTEND, DEBIAN_PRIORITY, APT_LISTCHANGES_FRONTEND)
  - Опции dpkg (--force-confdef, --force-confold, --force-confmiss)
  - Интерактивное распознавание запросов (русский и английский языки)
  - Увеличенный буфер для многострочных запросов (openssl и др.)
- ✅ **Управление репозиториями** — проверка, обновление, активация/деактивация через GUI
- ✅ **Режим тестирования (dry-run)** — безопасная проверка без изменений системы

Управление компонентами

- ✅ **Установка Wine** — автоматическая установка Wine для совместимости приложений
- ✅ **Установка Astra.IDE** — автоматическая установка среды разработки
- ✅ **Установка winetricks библиотек** — dotnet48, vcrun2013, vcrun2022 и др.
- ✅ **Установка Windows приложений** — через Wine (notepad++, winrar и др.)
- ✅ **Выбор компонентов** — чекбоксы в таблице для гибкого выбора устанавливаемых/удаляемых компонентов
- ✅ **Удаление компонентов** — безопасное удаление Wine, WINEPREFIX, Astra.IDE и связанных файлов
- ✅ **Автоматическое управление зависимостями** при удалении компонентов

Мониторинг и управление

- ✅ **Мониторинг процесса установки** в реальном времени:
  - Прогресс-бар с процентами
  - Время выполнения (мин/сек)
  - Размер установленных файлов (MB)
  - CPU: ▓▓▓

Технические возможности

- ✅ **Автоматическое обновление Python** — для работы графического интерфейса
- ✅ **Единый лог-файл** — все операции записываются в один файл через DualStreamLogger
- ✅ **Кнопка "Остановить"** — корректное прерывание процессов
- ✅ **Улучшенный терминал** — цветовая индикация и структурированный вывод
- ✅ **Автоматическое определение домашней директории** реального пользователя (при запуске через sudo)

🏗️ Архитектура системы

Общая архитектура (33 класса)

1. Архитектурные классы (3)

- **UniversalProcessRunner** — универсальный обработчик процессов для перенаправления всех коммуникационных действий в проекте
- **DualStreamLogger** — управление двумя потоками логирования (analysis и raw)
- **TerminalRedirector** — перенаправление sys.stdout/stderr

2. Базовые классы и обработчики компонентов (10)

- **ActivityTracker** — отслеживание активности классов
- **ComponentHandler** (ABC) — базовый класс для обработчиков компонентов
- **WinePackageHandler** — Wine пакеты (wine_astraregul, wine_9)
- **SystemConfigHandler** — системные настройки (ptrace_scope)
- **WineEnvironmentHandler** — Wine окружение (WINEPREFIX)
- **WinetricksHandler** — winetricks компоненты (dotnet48, vcrun2013, vcrun2022, и т.д.)
- **AptPackageHandler** — APT пакеты Linux (wine_system)
- **WineApplicationHandler** — Wine приложения Windows (notepad++, winrar)
- **ApplicationHandler** — приложения (Astra.IDE, CONT-Designer)
- **DesktopShortcutHandler** — ярлыки рабочего стола

3. Классы управления и мониторинга (7)

- **ComponentStatusManager** — менеджер статусов компонентов
- **ComponentInstaller** — координатор установки/удаления компонентов
- **SystemUpdater** — обновление Linux пакетов
- **SystemUpdateParser** — парсер обновлений системы
- **UniversalProgressManager** — универсальный менеджер прогресса
- **ProcessMonitor** — мониторинг процессов приложения
- **InstallationMonitor** — мониторинг установки

4. Классы проверки и валидации (4)

- **WineComponentsChecker** — проверка Wine компонентов
- **RepoChecker** — проверка репозиториев

- **DirectoryMonitor** — мониторинг изменений в директории
- **DirectorySnapshot** — снимок состояния директории

5. Классы работы с Wine (2)

- **WinetricksManager** — управление winetricks
- **WineApplicationInstanceManager** — менеджер экземпляров Wine приложений

6. Классы GUI и интерфейса (3)

- **AutomationGUI** — графический интерфейс (8 вкладок)
- **ToolTip** — всплывающие подсказки
- **InteractiveConfig** — конфигурация интерактивных запросов

7. Классы обработки интерактивных запросов (2)

- **InteractiveHandler** — обработка интерактивных запросов
- **LogReplaySimulator** — симулятор воспроизведения логов

8. Вспомогательные классы (2)

- **SystemStats** — анализ статистики системы
- **FilesystemFilter** — фильтр файловой системы

> Подробная информация о каждом классе доступна в файле `DocPlanes/PROJECT_ANALYSIS.md`

📦 Модульная структура проекта

Проект состоит из трех основных модулей, каждый из которых решает специфические задачи:

1. Основной модуль (astra_automation.py)

Назначение: Основной модуль системы автоматизации с графическим интерфейсом.

Функциональность:

- Автоматическое обновление Linux систем
- Управление компонентами (Wine, Astra.IDE, winetricks библиотеки)
- Графический интерфейс с 8 вкладками
- Мониторинг процессов и ресурсов
- Управление репозиториями APT

Размер: 26,757 строк кода

2. PackageManager — Модуль управления пакетами

Назначение: GUI инструмент для управления компонентами и создания архивов.

Расположение: `PackageManager/`

Основные возможности:

- ✅ Создание и управление компонентами
- ✅ Формирование архивов компонентов
- ✅ Работа с файловой структурой компонентов
- ✅ Создание сборок (bundles) компонентов
- ✅ Автоматическое создание шаблонов Wine приложений
- ✅ Параллельная архивация с прогрессом

Структура модуля:

```
PackageManager/
├── __init__.py          # Инициализация модуля
├── gui.py               # Основной GUI модуль (6,300 строк)
├── run_gui.py           # Скрипт запуска GUI
└── README.md            # Документация модуля
```

GUI вкладки (7 вкладок):

1. **Обзор** — просмотр структуры компонентов и архивов
2. **Создание компонента** — создание новых компонентов с конфигурацией
3. **Компоненты** — управление существующими компонентами
4. **Создание сборки** — формирование сборок из нескольких компонентов
5. **Структура папок** — управление структурой архивов
6. **Архивация** — массовая архивация компонентов
7. **Инструменты** — вспомогательные инструменты

Запуск:

```
# Способ 1: Через Python модуль
python3 -m PackageManager.run_gui

# Способ 2: Прямой запуск скрипта
python3 PackageManager/run_gui.py
```

> Подробная документация: `PackageManager/README.md`

3. DockerManager — Система сборки бинарников

Назначение: Универсальная система для автоматизации сборки бинарных файлов из Python проектов с использованием Docker.

Расположение: `DockerManager/`

Основные возможности:

- ✅ Локальная и удаленная сборка через Docker
- ✅ Сборка для разных платформ (Astra Linux 1.7, 1.8, Windows)

- ✅ Управление Docker образами
- ✅ Загрузка/скачивание исходников и результатов
- ✅ Графический интерфейс (GUI) для управления сборками
- ✅ Командная строка (CLI) для автоматизации
- ✅ Детальное логирование всех операций

Структура модуля:

```
DockerManager/
├── __init__.py              # Инициализация модуля
├── config.py                # Конфигурация (сервер, платформы, проекты)
├── cli.py                   # CLI интерфейс
├── build_manager_gui.py     # GUI приложение
├── build_runner.py          # Оркестратор сборки (локально/удаленно)
├── docker_manager.py        # Управление Docker образами
├── file_manager.py          # Загрузка/скачивание файлов
├── server_connection.py     # SSH/SCP соединение с сервером
├── logger.py                # Централизованное логирование
├── scripts/                 # Скрипты для Docker контейнеров
│   ├── docker_build.sh     # Скрипт сборки в контейнере
│   └── fix_future_imports.py # Исправление __future__ импортов
├── dockerfiles/             # Dockerfile для разных платформ
│   ├── Dockerfile.astra-1.7
│   └── Dockerfile.astra-1.8
├── run_gui.sh              # Запуск GUI (bash)
├── run_gui.py              # Запуск GUI (Python)
└── README.md               # Документация модуля
```

GUI вкладки (7 вкладок):

1. **Настройки сервера** — конфигурация удаленного сервера сборки
2. **Docker образы** — управление образами (список, создание, удаление)
3. **Исходники** — управление исходниками (загрузка, просмотр, очистка)
4. **Сборка** — запуск сборки (выбор проекта/платформы, прогресс)
5. **Результаты** — список сборок (фильтрация, скачивание, удаление)
6. **Лог сборки** — логи процесса сборки
7. **Лог работы** — реальное время логов всех операций

Запуск:

```
# GUI режим
python3 -m DockerManager.cli --gui

# CLI режим - локальная сборка
python3 -m DockerManager.cli --project FSA-AstraInstall --platform astra-
1.8

# CLI режим - удаленная сборка
```

```
python3 -m DockerManager.cli --project FSA-AstraInstall --platform astra-1.8 --remote
```

> Подробная документация: `DockerManager/README.md`

4. Build — Скрипты сборки проекта

Назначение: Скрипты для объединения файлов проекта и создания единого исполняемого файла.

Расположение: `Build/`

Структура модуля:

```
Build/
├── generate_unified.py     # Генерация объединенного файла
├── build_unified.py        # Объединение файлов проекта
├── self_updater.py         # Модуль самообновления
└── README.md               # Документация модуля
```

Процесс сборки:

1. **Генерация объединенного файла:**

   ```
   python3 Build/generate_unified.py
   ```

   Результат: `FSA-AstraInstall.py` в корне проекта

2. **Сборка бинарника (через DockerManager):**

   ```
   python3 -m DockerManager.cli --project FSA-AstraInstall --platform astra-1.8
   ```

   Результат: `FSA-AstraInstall` (бинарный файл) в корне проекта

Что объединяется:

- `astra_automation.py` — основной модуль
- `astra_update.sh` — скрипт обновления (встраивается как строка)
- `astra_install.sh` — скрипт установки (встраивается как строка)
- `self_updater.py` — модуль самообновления

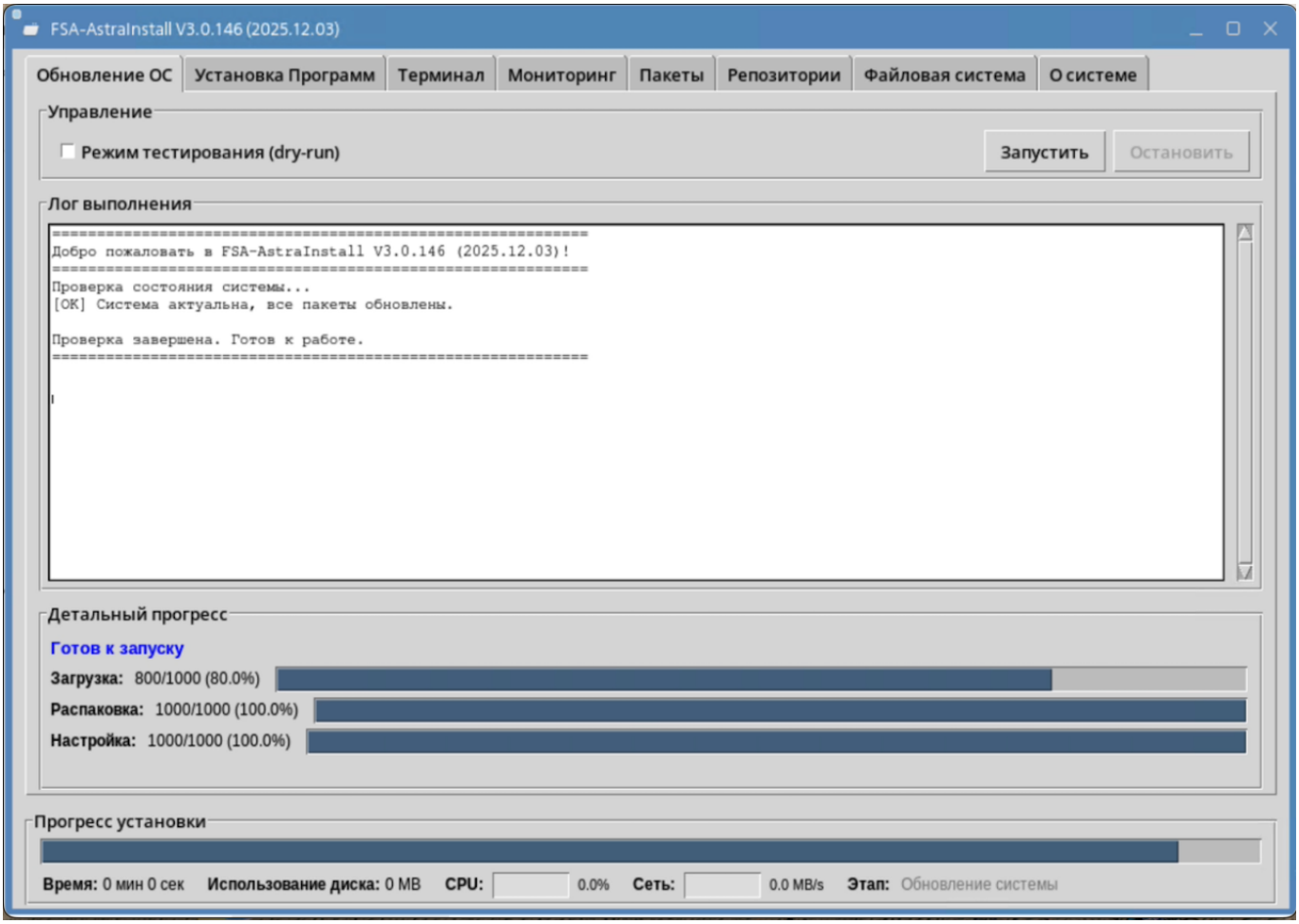
> Подробная документация: `Build/README.md`

🖥️ Графический интерфейс

Основное приложение (AutomationGUI)

Графический интерфейс основного приложения состоит из **8 вкладок**:

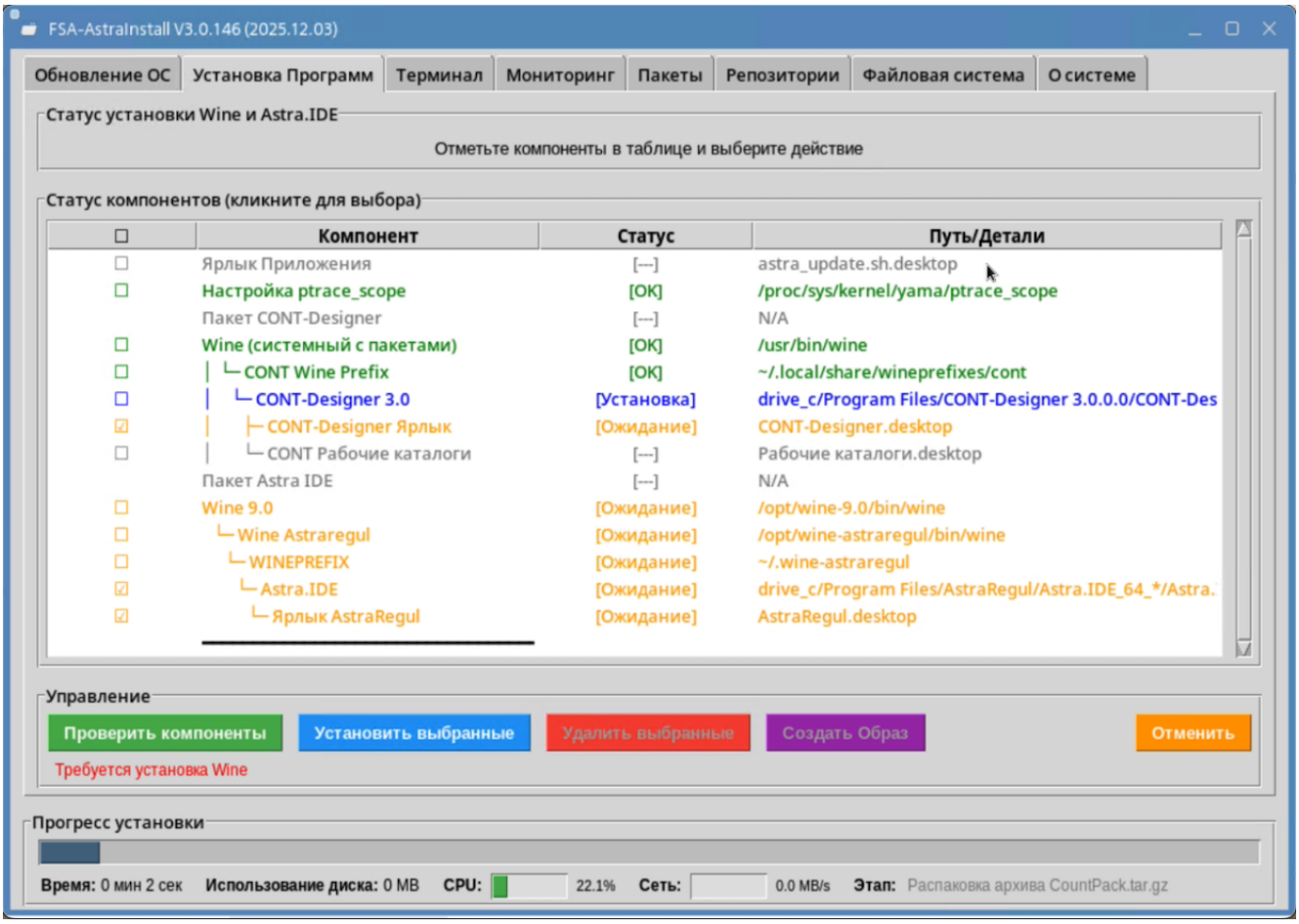
1. Вкладка "Обновление ОС" — главный экран



Функциональность:

- Режим тестирования (dry-run)
- Кнопки запуска и остановки
- Системный терминал для вывода операций
- Лог выполнения с возможностью сохранения
- Статистика репозиториев и пакетов

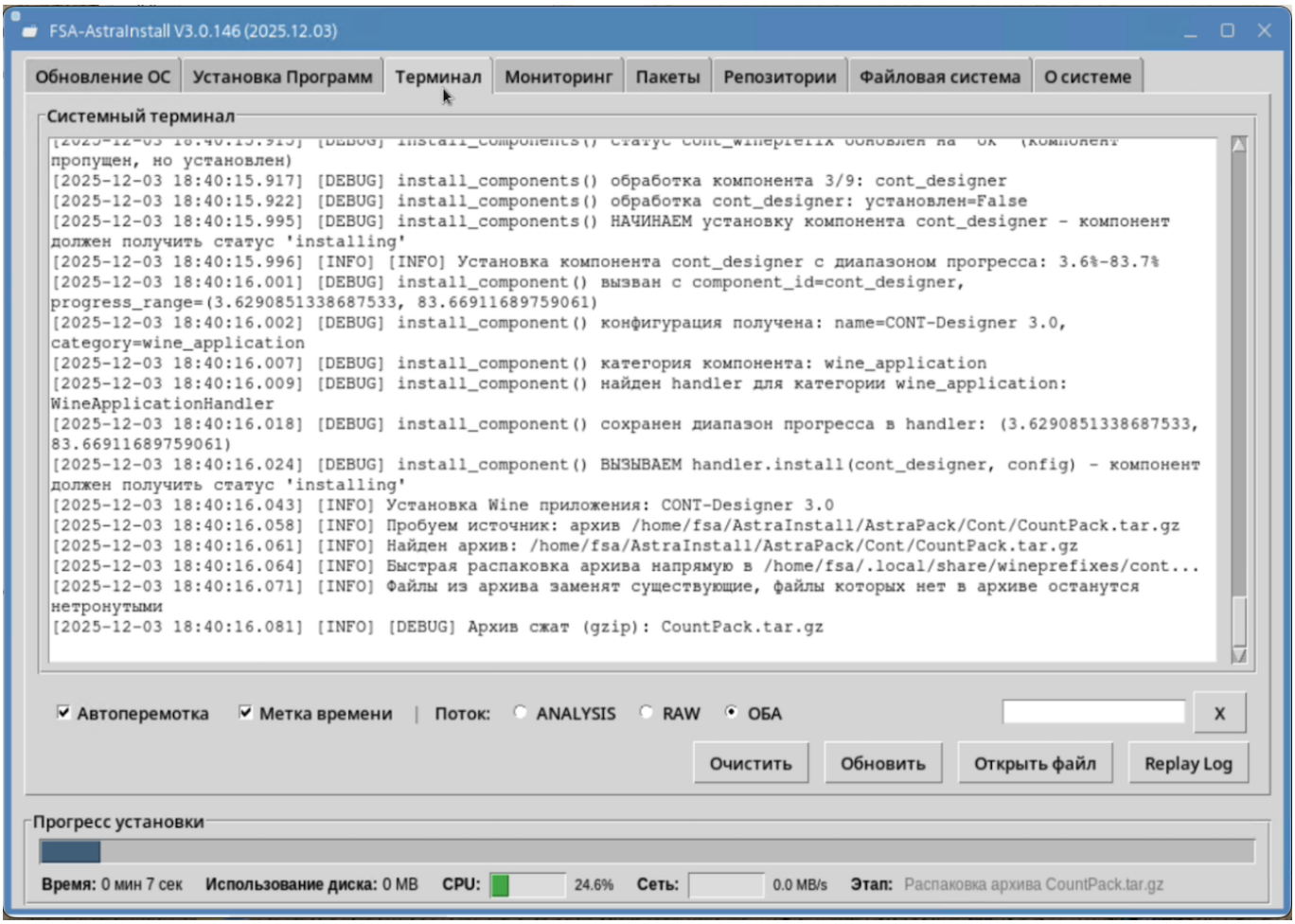
2. Вкладка "Установка Программ" — управление компонентами



Функциональность:

- Проверка статуса всех компонентов (Wine, WINEPREFIX, winetricks библиотеки, Linux пакеты)
- Выборочная установка/удаление через чекбоксы
- Мониторинг установки в реальном времени (время, размер, CPU, сеть, процессы Wine)
- Отображение путей к установленным компонентам
- Цветовая индикация статуса ([OK] зеленый, [ERR] красный)

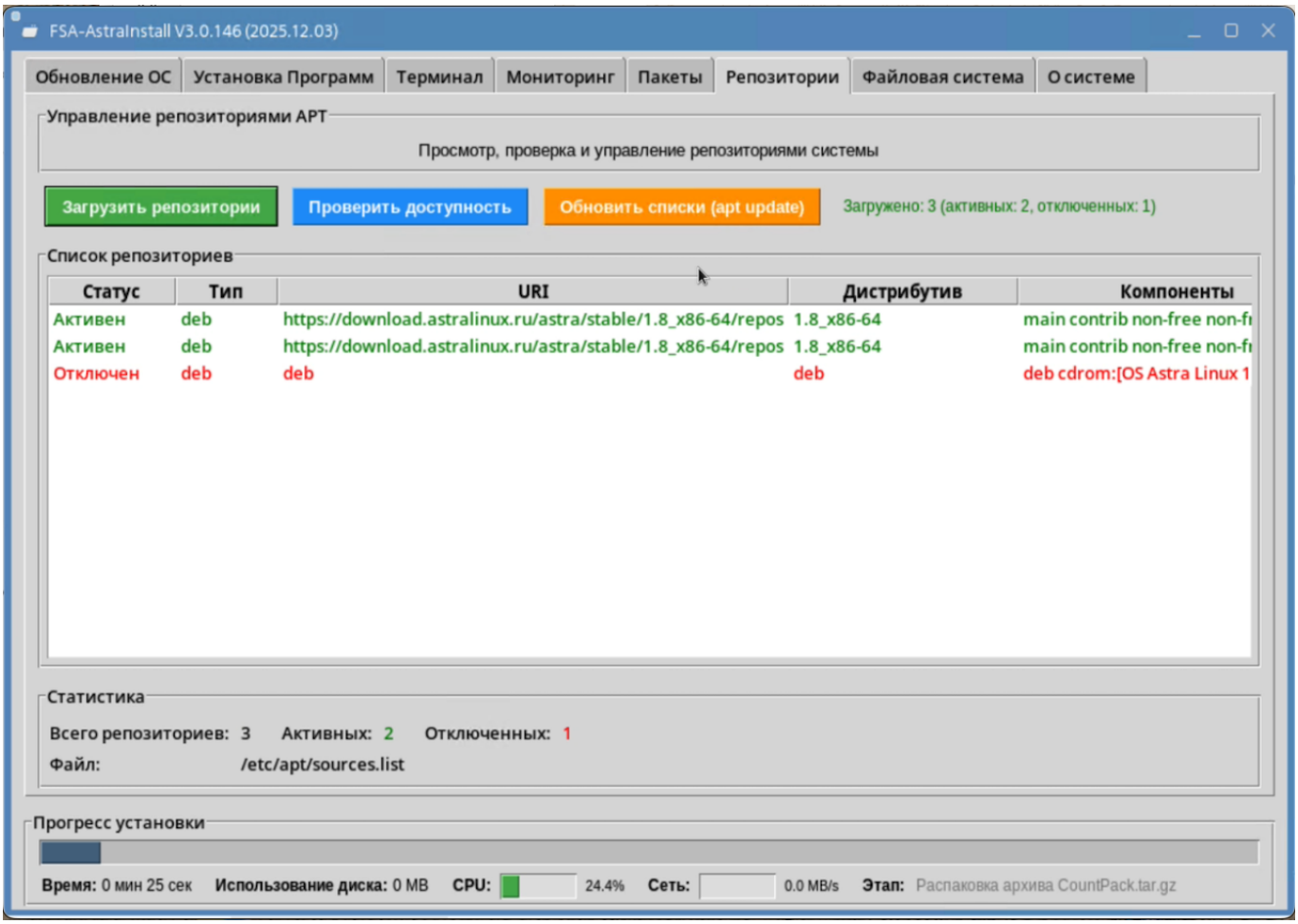
3. Вкладка "Терминал" — системный вывод



Функциональность:

- Перенаправление stdout/stderr
- Копирование текста
- Автоматическая прокрутка
- Сохранение логов в папку Log/
- Черный фон для лучшей читаемости

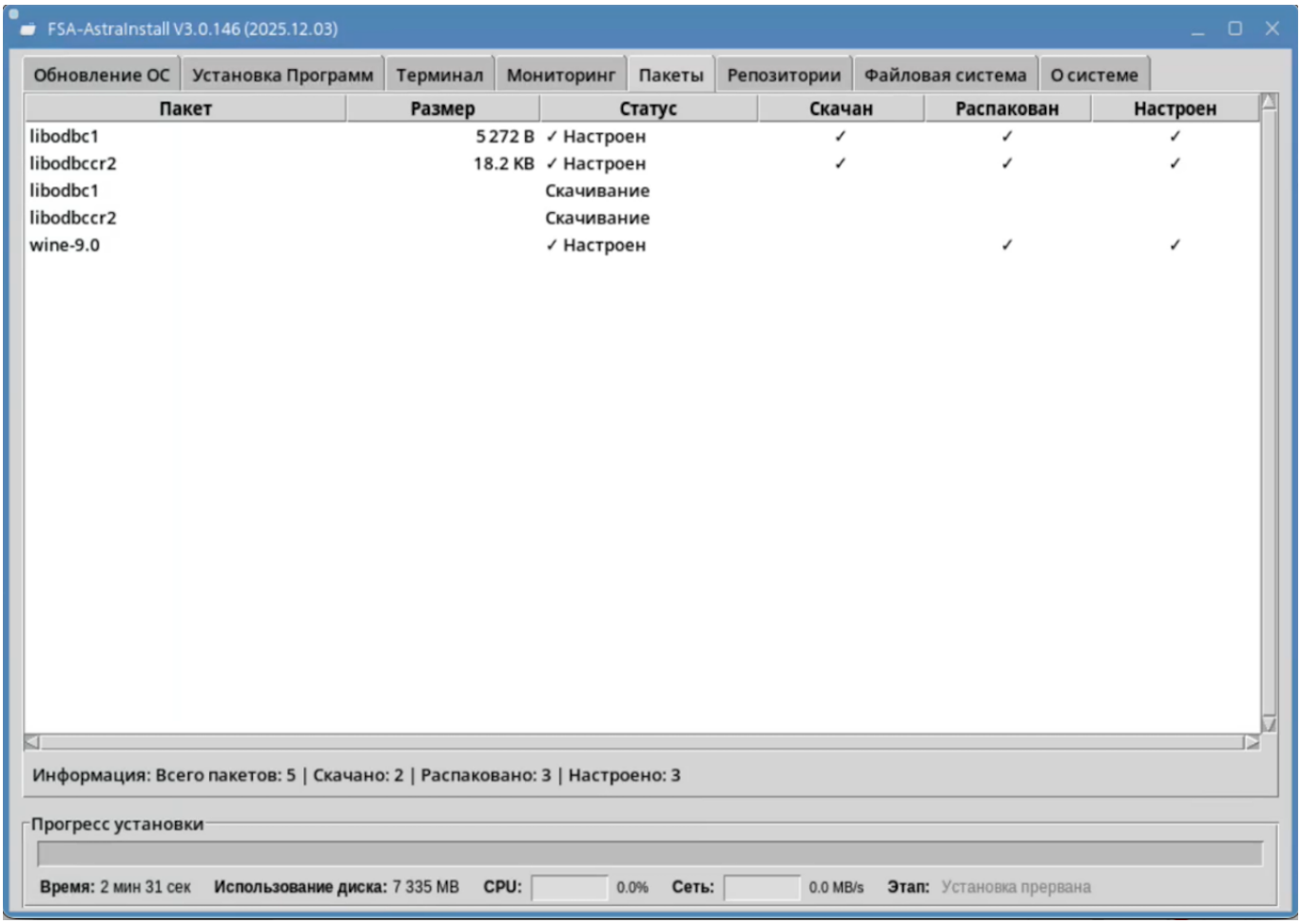
4. Вкладка "Мониторинг" — мониторинг процессов



Функциональность:

- Отслеживание активных процессов
- Мониторинг ресурсов (CPU, память)
- Отслеживание процессов Wine
- Информация о запущенных приложениях

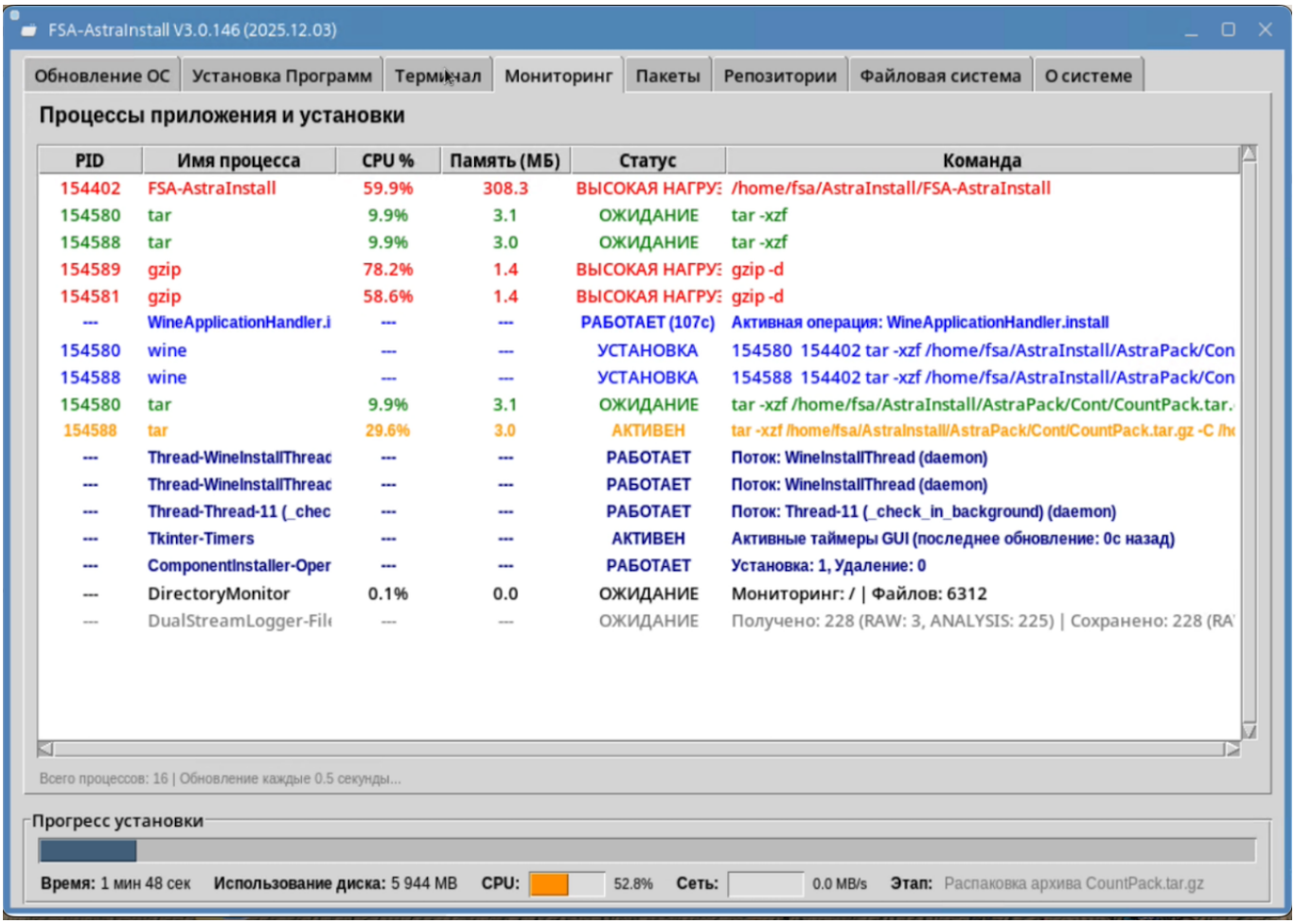
5. Вкладка "Пакеты" — управление пакетами



Функциональность:

- Просмотр установленных пакетов
- Поиск пакетов
- Информация о пакетах
- Управление пакетами

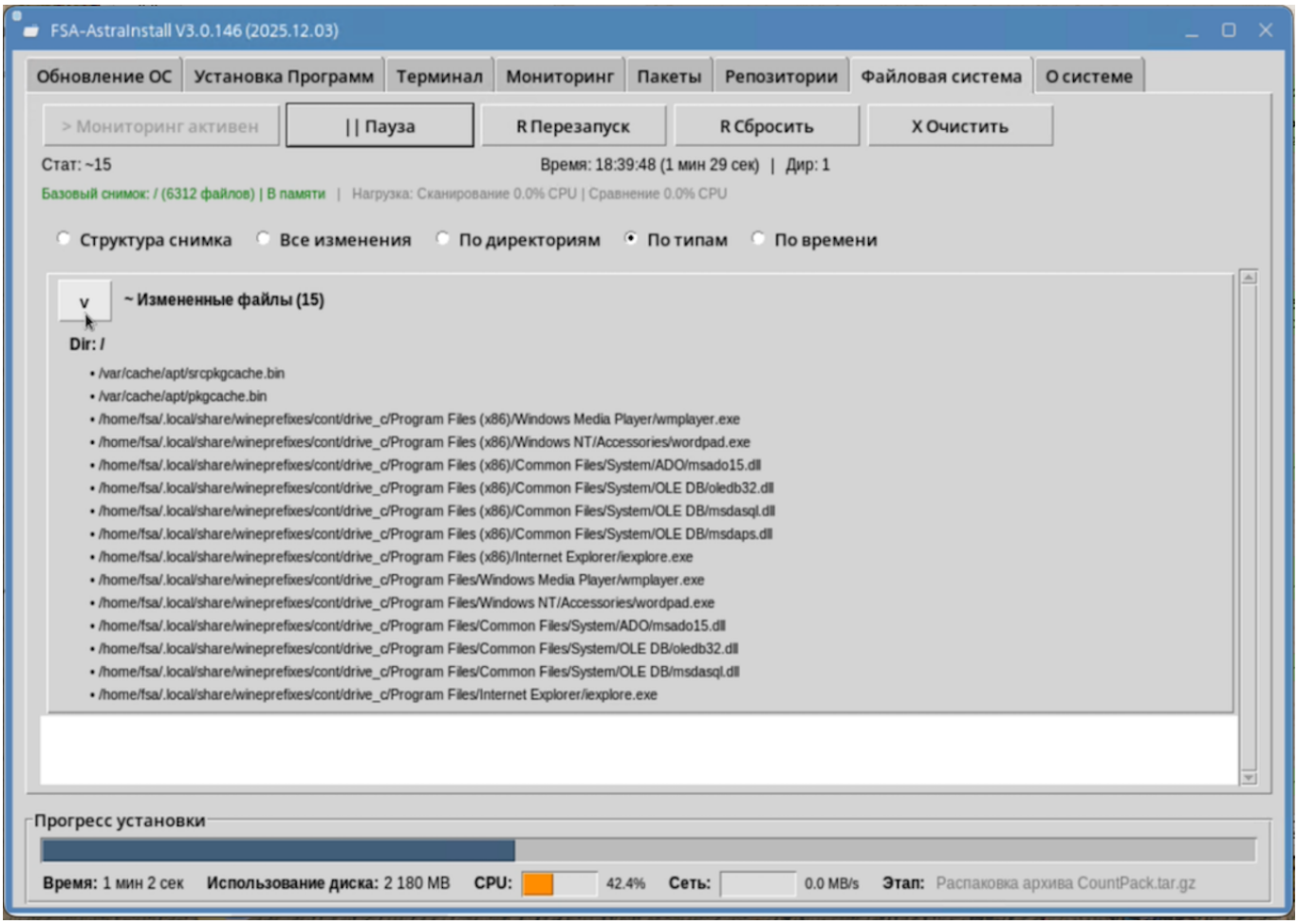
6. Вкладка "Репозитории" — управление источниками пакетов



Функциональность:

- Просмотр всех репозиториев системы
- Проверка доступности каждого репозитория
- Активация/деактивация репозиториев
- Обновление списков пакетов (apt update)
- Статистика по активным/отключенным репозиториям

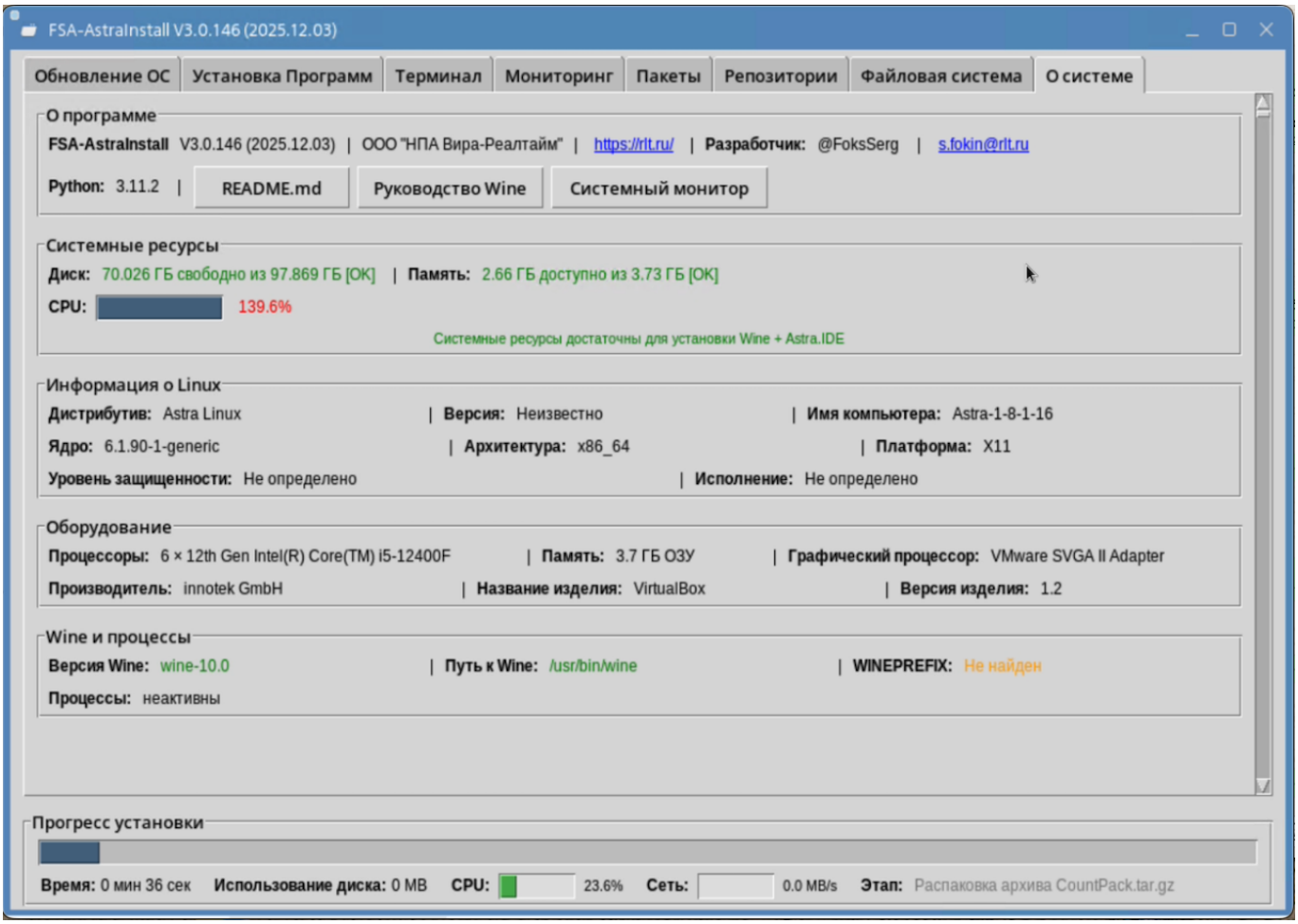
7. Вкладка "Файловая система" — мониторинг файлов



Функциональность:

- Отслеживание изменений в директориях
- Снимки состояния файлов
- Анализ изменений
- История модификаций

8. Вкладка "О системе" — информация о системе



Функциональность:

- Версия системы
- Информация о процессоре
- Информация о памяти
- Информация о дисках
- Сетевая информация

🔧 Система сборки и развертывания

Процесс сборки бинарника

Бинарный файл собирается через Docker на удалённом сервере (Windows + WSL2) или локально.

Этап 1: Генерация объединенного файла

```
python3 Build/generate_unified.py
```

Результат: `FSA-AstraInstall.py` (1.6 MB, 29,791 строк)

Этап 2: Сборка бинарника

Локальная сборка:

```
python3 -m DockerManager.cli --project FSA-AstraInstall --platform astra-1.8
```

Удаленная сборка:

```
python3 -m DockerManager.cli --project FSA-AstraInstall --platform astra-1.8 --remote
```

Процесс сборки:

1. Проверка объединенного файла (`FSA-AstraInstall.py`)
2. Получение Dockerfile для платформы
3. Проверка/сборка Docker образа
4. Запуск контейнера с монтированием проекта
5. Выполнение скрипта сборки в контейнере (PyInstaller)
6. Копирование результата из контейнера
7. Очистка временных контейнеров

Результат: `FSA-AstraInstall` (11.8 MB, автономный исполняемый файл)

Поддерживаемые платформы:

- Astra Linux 1.7 (GLIBC 2.28)
- Astra Linux 1.8 (GLIBC 2.36)
- Windows (планируется)

Требования для сборки

Локальная сборка:

- macOS (для запуска DockerManager)
- Docker Desktop
- Python 3.7+

Удаленный сервер:

- Linux (Ubuntu/Debian/WSL2)
- Docker
- SSH сервер
- Достаточно места на диске для образов и сборок

📦 Управление компонентами

Система компонентов

Проект использует модульную систему компонентов, каждый из которых может быть:

- **Wine пакеты** — wine_astraregul, wine_9
- **Wine окружение** — WINEPREFIX с настройками
- **Winetricks библиотеки** — dotnet48, vcrun2013, vcrun2022
- **Linux пакеты** — wine_system и другие APT пакеты
- **Wine приложения** — notepad++, winrar
- **Приложения** — Astra.IDE, CONT-Designer
- **Ярлыки рабочего стола** — автоматическое создание ярлыков

PackageManager — инструмент управления

Назначение: Создание, управление и архивация компонентов.

Основные функции:

1. **Создание компонентов** — создание новых компонентов с конфигурацией
2. **Управление компонентами** — просмотр, редактирование, удаление
3. **Создание сборок** — формирование сборок из нескольких компонентов
4. **Архивация** — массовая архивация компонентов в tar.gz архивы
5. **Структура папок** — управление структурой архивов компонентов

Расположение архивов: `AstraPack/`

Формат компонентов:

- Каждый компонент хранится в отдельном tar.gz архиве
- Архив содержит файлы компонента и конфигурацию (JSON)
- Конфигурация описывает тип компонента, зависимости, пути установки

🚀 Использование

Рекомендуемый способ: Бинарный файл

```
# Скопируйте FSA-AstraInstall на Linux машину
# Сделайте исполняемым (один раз)
chmod +x FSA-AstraInstall

# Запустите (автоматически запросит sudo при необходимости)
./FSA-AstraInstall
```

Преимущества бинарника:

- ✅ Не требует установки Python
- ✅ Автономный файл — всё включено
- ✅ Автоматический запрос прав root
- ✅ Работает на любой совместимой системе

Альтернатива: Bash-скрипт

```
# Скопируйте проект на Linux машину
# Запустите скрипт - он всё сделает сам!
bash astra_install.sh
```

Этот скрипт:

- ✅ Автоматически запросит права root через `sudo`
- ✅ Проверит что нужно для GUI
- ✅ Автоматически установит недостающие компоненты
- ✅ Запустит графическую форму для настройки
- ✅ Выполнит автоматическое обновление системы
- ✅ Установит Wine и Astra.IDE

Альтернативные режимы

Консольный режим (без GUI):

```
bash astra_install.sh --console
```

Режим тестирования (dry-run):

```
bash astra_install.sh --dry-run
```

Консольный режим с тестированием:

```
bash astra_install.sh --console --dry-run
```

Режим тестирования (dry-run)

Что делает dry-run:

- ✅ **Проверяет** доступность репозиториев
- ✅ **Показывает** какие репозитории будут активированы/деактивированы
- ✅ **Симулирует** обновление системы
- ✅ **Анализирует** статистику пакетов
- ❌ **НЕ изменяет** системные файлы
- ❌ **НЕ устанавливает** пакеты
- ❌ **НЕ обновляет** систему

Когда использовать:

- Перед реальной установкой
- Для проверки доступности репозиториев

- Для анализа что будет установлено
- Для тестирования на критических системах

⚙️ Технические требования

Требования для запуска

- **ОС:** Linux x86-64 (Ubuntu, Debian, Astra Linux и другие дистрибутивы)
- **Python:** Python 3.x (обновляется автоматически, если используется скрипт)
- **Права:** root для установки пакетов (запрашивается автоматически)
- **Сеть:** Доступ к репозиториям системы
- **GLIBC:** 2.28+ (автоматически выполняется на всех поддерживаемых системах)

Компоненты Python (устанавливаются автоматически)

Для работы GUI устанавливается:

- **python3** — обновленный интерпретатор
- **python3-dev** — заголовки для разработки
- **python3-pip** — менеджер пакетов
- **python3-setuptools** — инструменты сборки
- **python3-distutils** — инструменты дистрибуции
- **python3-tk** — Tkinter для GUI
- **python3-venv** — виртуальные окружения

Процесс автоматизации

1. **Проверка системы** — анализ текущего состояния
2. **Настройка репозиториев** (bash скрипт) — проверка и активация рабочих репозиториев
3. **Обновление Python** — если необходимо для GUI
4. **Запуск GUI** — графический интерфейс для управления (только в GUI режиме)
5. **Обновление системы** (Python скрипт) — из настроенных репозиториев
6. **Автоподтверждение** — автоматические ответы на запросы
7. **Установка Wine** — для совместимости приложений
8. **Установка дополнительных компонентов** — среды разработки и прочие компоненты
9. **Управление компонентами** — возможность переустановки/удаления
10. **Завершение** — отчет о результатах

Важно: Настройка репозиториев выполняется один раз в bash скрипте перед запуском Python. Python скрипт использует уже настроенные репозитории для обновления системы.

🔍 Устранение проблем

Репозитории недоступны

```
# Проверьте интернет соединение
ping google.com
```

```
# Проверьте доступность репозиториев вашей системы
ping archive.ubuntu.com  # для Ubuntu
ping ftp.astralinux.ru   # для Astra Linux
ping mirror.yandex.ru    # для других дистрибутивов

# Если репозитории недоступны, используйте режим тестирования
sudo python3 astra_automation.py --console --dry-run
```

Python не обновляется

```
# Исправьте зависимости
sudo apt-get install -f

# Обновите вручную
sudo apt-get install python3
```

GUI не запускается

```
# Установите Tkinter
sudo apt-get install python3-tk

# Проверьте установку
python3 -c "import tkinter"
```

Решение проблемы Python/Tkinter

Для работы графического интерфейса может потребоваться обновление Python:

Проблема:

- Базовая версия Astra Linux может иметь устаревший Python 3
- Для GUI нужен обновленный Python 3 с Tkinter
- Репозитории Astra Linux недоступны извне (403 Forbidden)

Решение:

- Скрипты для обновления Python 3 через `apt-get` на Linux машине
- Автоматическая настройка репозиториев Astra Linux
- Установка Python 3 + Tkinter + все зависимости
- После обновления Python запускается графический интерфейс

📚 Поддержка и документация

Документация проекта

- **README.md** (этот файл) — основная документация проекта
- **DocPlanes/PROJECT_ANALYSIS.md** — подробный анализ классов и модулей
- **PackageManager/README.md** — документация модуля управления пакетами
- **DockerManager/README.md** — документация системы сборки
- **Build/README.md** — документация скриптов сборки
- **DocInstruction/** — инструкции по использованию

Структура проекта

```
FSA-AstraInstall/
├── FSA-AstraInstall             # БИНАРНИК - автономный исполняемый файл
(11.8 MB)
├── FSA-AstraInstall.py          # Объединённый Python-файл (1.6 MB,
29,791 строк)
├── README.md                    # Документация проекта (этот файл)
├── astra_install.sh             # Bash-скрипт запуска (995 строк)
├── astra_automation.py          # Основной модуль с GUI (26,757 строк)
├── Build/                       # Скрипты сборки
│   ├── generate_unified.py     # Генерация объединенного файла
│   ├── build_unified.py        # Объединение файлов проекта
│   ├── self_updater.py         # Модуль самообновления
│   └── README.md               # Документация модуля
├── PackageManager/             # Модуль управления пакетами
│   ├── gui.py                  # GUI модуль (6,300 строк)
│   ├── run_gui.py              # Скрипт запуска GUI
│   └── README.md               # Документация модуля
├── DockerManager/               # Система сборки бинарников
│   ├── build_manager_gui.py    # GUI приложение
│   ├── build_runner.py         # Оркестратор сборки
│   ├── docker_manager.py       # Управление Docker образами
│   ├── file_manager.py         # Загрузка/скачивание файлов
│   ├── server_connection.py    # SSH/SCP соединение
│   ├── cli.py                  # CLI интерфейс
│   ├── config.py               # Конфигурация
│   ├── dockerfiles/            # Dockerfile для платформ
│   └── README.md               # Документация модуля
├── DocPlanes/                   # Планы и анализ проекта
│   ├── PROJECT_ANALYSIS.md     # Подробный анализ классов
│   └── README.md               # Описание планов
├── DocInstruction/              # Инструкции по использованию
│   ├── STARTUP_ALGORITHM.md    # Алгоритм запуска
│   ├── WINE_INSTALL_GUIDE.md   # Руководство по Wine
│   └── README.md               # Описание инструкций
├── Screenshots/                 # Скриншоты интерфейса
├── AstraPack/                   # Архивы компонентов
├── version.txt                  # Версия проекта
└── History/                     # Локальная история версий
```

Поддержка

Если у вас возникли проблемы:

1. Запустите тесты: `python3 test_complete.py`
2. Проверьте режим тестирования: `sudo python3 astra_automation.py --console --dry-run`
3. Проверьте логи: `sudo apt-get update`
4. Обратитесь к администратору системы
5. Проверьте документацию Astra Linux

📝 Система версионирования

Проект использует двойную систему версионирования:

🏷️ Версия проекта: VMAJOR.MINOR.PATCH

- **MAJOR** (VMAJOR.X.X) — кардинальные изменения архитектуры
- **MINOR** (X.MINOR.X) — новые функции, улучшения
- **PATCH** (X.X.PATCH) — количество коммитов (сохраненных изменений)

📅 Дата релиза файла: YYYY.MM.DD

- Индивидуальная для каждого файла
- Обновляется только при изменении конкретного файла

📝 Правила обновления:

1. **При изменении функциональности проекта** → обновляем версию проекта во ВСЕХ файлах
2. **При изменении конкретного файла** → обновляем дату релиза ТОЛЬКО в этом файле
3. **При изменении правил ASSISTANT_RULES.md** → обновляем версию проекта во ВСЕХ файлах
4. **MAJOR версия** (VMAJOR.X.X) — кардинальные изменения архитектуры
5. **MINOR версия** (X.MINOR.X) — новые функции, улучшения
6. **PATCH версия** (X.X.PATCH) — номер коммита (сохраненных изменений)

🔮 Планы развития

- 🔄 Автоматическая установка дополнительных Linux приложений
- 🔄 Автоматическая установка Windows приложений через Wine
- 🔄 Создание каталога популярных приложений для быстрой установки
- 🔄 Поддержка разных дистрибутивов Linux (CentOS, Fedora, Arch)
- 🔄 Веб-интерфейс для удаленного управления
- 🔄 Система плагинов для расширения функциональности
- 🔄 Автоматическое создание резервных копий системы
- 🔄 Мониторинг производительности и ресурсов системы
- 🔄 Расширение поддержки различных Linux дистрибутивов

📄 Лицензия

Проект разработан **ООО "НПА Вира-Реалтайм"**

Разработчик: @FoksSegr & AI Assistant (@LLM)

Версия документа: V3.0.146 (2025.12.03)